Hexaware Technologies — Azure Cloud Engineer

L1 Interview Preparation Handbook | Landing Zone + Networking + AKS + Python + Terraform + SRE

Target profile: 9–12 years | Azure Cloud Engineering | Enterprise Landing Zones | Azure Networking | AKS/Kubernetes | Python Automation | Terraform | CI/CD

How to Use This Handbook

• Answer directly first, then give one practical production example.

• For troubleshooting use: Impact → Triage → Metrics/Logs → Isolate → Mitigate → Validate → RCA → Prevent.

• Do not claim ownership of architecture you did not actually implement. Clearly distinguish hands-on work from conceptual/design knowledge.

• For Python questions, emphasize cloud automation, APIs, idempotency, exception handling, retries, logging and secure authentication.

• For AKS questions, discuss production concerns: scaling, networking, identity, observability, upgrades and reliability.

1. Opening and Profile

Q: Tell me about yourself.

Answer: I have around 11 years of experience in Cloud, SRE and DevOps, with strong experience across Azure, Kubernetes, Terraform, CI/CD and production operations. In my recent project, I worked on Azure Synapse Analytics, supporting production workloads across multiple regions. My responsibilities included monitoring and observability, incident management, SLI/SLOs, reliability improvements, capacity planning, CI/CD and automation. I have experience with Azure services, Kubernetes/AKS concepts, Docker, Terraform, Azure DevOps, Prometheus, Grafana, Azure Monitor and scripting with Python, Bash and PowerShell. My focus is automation, reliability, troubleshooting and reducing operational toil.

Q: Why are you interested in this Azure Cloud Engineer role?

Answer: The role combines the areas I have been working with—Azure, Kubernetes, Terraform, automation, CI/CD and production reliability. I am particularly interested in the combination of enterprise Azure foundations, networking, AKS and Python automation because it allows me to apply both cloud engineering and SRE practices.

Q: Describe your recent Azure project.

Answer: My recent project was around Azure Synapse Analytics, where I worked as part of the SRE function supporting production workloads across multiple regions. My responsibilities included production monitoring, P1/P2 incident handling, dashboards and logs, alert optimization, SLI/SLO work, CI/CD, automation and capacity/reliability planning. I collaborated with engineering and platform teams during incidents and production changes.

2. Azure Landing Zone and Governance

Q: What is an Azure Landing Zone?

Answer: An Azure Landing Zone is a standardized foundation for deploying and operating workloads in Azure at enterprise scale. It establishes the structure for subscriptions, management groups, identity, RBAC, networking, security, policies, logging and governance before application workloads are deployed.

Q: Explain an enterprise Azure Landing Zone architecture.

Answer: I would organize subscriptions under management groups based on platform and workload requirements. Platform-oriented subscriptions can host connectivity, identity and management capabilities, while application subscriptions host workload resources. A connectivity subscription can contain a hub VNet, Azure Firewall, VPN/ExpressRoute connectivity and shared DNS services. Governance is applied with Azure Policy, RBAC, tagging, diagnostics and centralized monitoring.

Q: Management Groups vs Subscriptions vs Resource Groups?

Answer: Management Groups organize subscriptions and provide a hierarchy for governance. A Subscription is a billing, isolation and resource-management boundary. A Resource Group is a logical container for resources that generally share a lifecycle. Policies can be applied at management-group or subscription scope and inherited downward.

Q: What is Azure Policy?

Answer: Azure Policy is a governance mechanism used to audit or enforce organizational standards. Examples include allowed regions, required tags, restricted public IPs, diagnostic settings, encryption and approved SKUs. Policy answers what configurations are allowed or compliant; RBAC answers who can perform actions.

Q: Azure Policy vs RBAC?

Answer: RBAC controls authorization—who can do what at a particular scope. Azure Policy controls resource configuration and compliance—what is allowed or required. For example, RBAC may give a DevOps identity Contributor access to a resource group, while Policy may prevent deployment of resources outside approved regions.

Q: What is an Azure Policy Initiative?

Answer: An Initiative is a collection of related policy definitions managed as a group. For example, a security initiative could contain policies requiring encryption, approved regions, diagnostic logging and tags.

Q: Explain Azure RBAC.

Answer: Azure RBAC is authorization based on a security principal, a role definition and a scope. The scope can be a management group, subscription, resource group or individual resource. I prefer least-privilege access and avoid unnecessarily broad Owner or Contributor permissions.

Q: What is Managed Identity and why use it?

Answer: Managed Identity provides an Azure-managed identity for a resource so applications can authenticate to Azure services without storing credentials in code or configuration. It reduces secret-management overhead and improves security. Workload Identity can similarly allow Kubernetes workloads to access Azure resources using federated identity.

Q: What is Azure Key Vault?

Answer: Key Vault stores and controls access to secrets, keys and certificates. Applications should retrieve secrets securely rather than hardcoding them in source control, Terraform configuration, container images or Kubernetes manifests. Access should use managed identity or workload identity and least-privilege authorization.

Q: Have you implemented Azure Landing Zones?

Answer: I have worked extensively with the Azure components and governance concepts that form an enterprise Landing Zone, including subscriptions, networking, RBAC, policies, private connectivity, monitoring and infrastructure automation. My strongest hands-on experience is in Azure production operations, SRE, Kubernetes and automation. For a greenfield enterprise Landing Zone, I would use Microsoft's recommended architecture, Terraform modules and a phased platform-and-workload model.

3. Azure Networking

Q: Explain hub-and-spoke architecture.

Answer: The hub is the centralized networking layer and spokes contain individual application workloads. The hub can host Azure Firewall, VPN Gateway, ExpressRoute Gateway, Bastion and shared DNS services. Application VNets are spokes connected to the hub using VNet peering. This centralizes common security and connectivity while isolating application environments.

Q: What is VNet peering?

Answer: VNet peering provides private connectivity between Azure VNets over Microsoft's backbone network. Regional and global peering are available. Peering is not automatically transitive, so two spokes do not communicate merely because both are peered with a hub; routing and security controls must be designed explicitly.

Q: What is an NSG?

Answer: A Network Security Group contains inbound and outbound rules based on source, destination, port, protocol, direction and priority. It can be associated with subnets and network interfaces. Lower numerical priority values are evaluated before higher ones.

Q: NSG vs Azure Firewall?

Answer: NSGs provide distributed subnet/NIC-level traffic filtering. Azure Firewall is a centralized managed firewall service for broader network security and traffic control. In a hub-spoke design, I would commonly use NSGs for workload segmentation and Azure Firewall for centralized inspection and controlled ingress/egress paths.

Q: What is a Private Endpoint?

Answer: A Private Endpoint provides a private IP address in a VNet for access to an Azure PaaS service through Azure Private Link. It enables private connectivity without requiring the application to use the service's public endpoint. Private DNS must often be configured correctly so the service hostname resolves to the private IP.

Q: Private Endpoint vs Service Endpoint?

Answer: A Service Endpoint extends VNet identity to an Azure service while the service continues to use its public endpoint. A Private Endpoint creates a private IP in the VNet for the service. Private Endpoint provides stronger private-connectivity isolation and is commonly preferred when eliminating public exposure is a requirement.

Q: Azure Load Balancer vs Application Gateway?

Answer: Azure Load Balancer is primarily a Layer 4 service for TCP/UDP load balancing. Application Gateway operates at Layer 7 for HTTP/HTTPS and supports capabilities such as host-based and URL-based routing, TLS termination and WAF integration.

Q: VPN vs ExpressRoute?

Answer: A site-to-site VPN provides encrypted connectivity between on-premises and Azure over the internet. ExpressRoute provides private connectivity through an ExpressRoute provider. ExpressRoute is commonly used where enterprise requirements call for predictable private connectivity, performance and higher availability.

Q: What is a route table / UDR?

Answer: A user-defined route table contains custom routes that influence how traffic is forwarded. It is commonly used in hub-spoke architectures to force traffic through an NVA or Azure Firewall, or to control paths to on-premises networks.

Q: What is Azure DNS / Private DNS?

Answer: Azure DNS hosts DNS zones. Private DNS zones provide name resolution inside VNets for private resources such as Private Endpoints. Correct private DNS links and records are critical when applications must resolve an Azure PaaS hostname to a private IP.

Q: How would you troubleshoot an Azure workload that cannot access Storage privately?

Answer: I would establish scope and impact first. Then I would verify DNS resolution, routing and connectivity from the source. I would check NSGs, route tables, firewall rules and Private Endpoint configuration. For Private Endpoint, I would specifically verify the private DNS zone and confirm the hostname resolves to the expected private IP. I would use Network Watcher and relevant logs where appropriate, then validate connectivity from the application after remediation.

Q: How would you troubleshoot Azure network connectivity?

Answer: I work from the source to the destination: DNS → IP → route → NSG → firewall → subnet/VNet → destination. I verify whether the expected private or public path is being used, inspect effective routes and security rules, test connectivity from the actual workload, and compare the timing with recent infrastructure changes.

4. AKS and Kubernetes

Q: Explain AKS architecture.

Answer: AKS is Microsoft's managed Kubernetes service. Azure manages the Kubernetes control plane, while worker nodes and workloads are managed by the customer depending on the configuration. The control plane includes the API server, scheduler, controllers and etcd. Worker nodes run kubelet, the container runtime and workloads. Production design also considers node pools, networking, identity, monitoring, upgrades, availability zones and autoscaling.

Q: Why use multiple AKS node pools?

Answer: Multiple node pools provide workload isolation and allow different VM sizes and scaling strategies. Examples include system nodes, general application nodes, memory-optimized nodes or GPU nodes. Taints, labels and affinity can control which workloads are scheduled onto each pool.

Q: Azure CNI vs Kubenet?

Answer: With Azure CNI, pods receive IP addresses from Azure VNet/subnet address space, providing direct Azure network integration. Kubenet uses a separate pod address space with routing/NAT to integrate pod traffic with the VNet. The choice affects IP planning, networking behavior and scalability considerations.

Q: What is a Kubernetes Pod?

Answer: A Pod is the smallest deployable unit in Kubernetes and contains one or more containers that share network and storage context. Most application Pods contain one main container, while sidecar patterns can add supporting containers.

Q: Deployment vs StatefulSet?

Answer: Deployments are commonly used for stateless workloads and manage replicated Pods through ReplicaSets. StatefulSets are designed for stateful workloads requiring stable network identity, persistent storage and ordered deployment or scaling semantics.

Q: What is a Kubernetes Service?

Answer: A Service provides a stable network endpoint for a set of Pods selected by labels. Common types are ClusterIP, NodePort and LoadBalancer. ClusterIP is internal to the cluster, while LoadBalancer integrates with the cloud provider to expose a service externally.

Q: What is Ingress?

Answer: Ingress provides HTTP/HTTPS routing from outside the cluster to Kubernetes Services. It can route by host or path and can integrate with an ingress controller such as Application Gateway for Containers or another supported controller. TLS termination and WAF capabilities depend on the chosen architecture.

Q: What is Helm?

Answer: Helm is a package manager for Kubernetes. A chart packages Kubernetes resources such as Deployments, Services, ConfigMaps and Ingress templates. Common commands include helm install, helm upgrade, helm rollback, helm history and helm list.

Q: Explain HPA vs Cluster Autoscaler vs VPA.

Answer: HPA adjusts the number of Pods based on metrics. Cluster Autoscaler adjusts the number of worker nodes when scheduling demand requires more or fewer nodes. VPA adjusts resource recommendations or requests/limits based on workload usage, depending on how it is configured.

Q: Readiness vs Liveness vs Startup probes?

Answer: Liveness determines whether a container is healthy enough to continue running; failure can trigger a restart. Readiness determines whether the Pod should receive traffic; failure removes it from Service endpoints without necessarily restarting it. Startup probes are useful for slow-starting applications so liveness checks do not restart them prematurely.

Q: How do you troubleshoot CrashLoopBackOff?

Answer: I first determine impact and scope. I check kubectl get pods, then kubectl describe pod for events and kubectl logs, including --previous for the previous container instance. I look for application errors, configuration problems, missing Secrets, failed probes, OOMKilled events, dependency failures and recent changes. I mitigate with rollback or configuration/resource correction, validate recovery, and then perform RCA and prevention.

Q: How do you troubleshoot ImagePullBackOff?

Answer: I check the image name and tag, confirm the image exists in ACR or the configured registry, and inspect Pod events. I verify registry authentication, imagePullSecrets or the AKS identity's permissions, plus DNS/network connectivity to the registry. I also check whether a recent deployment introduced a bad tag.

Q: How do you troubleshoot a Pending Pod?

Answer: I inspect kubectl describe pod and the scheduler events. Common causes are insufficient CPU or memory, node selectors, taints/tolerations, affinity rules, unavailable nodes, PVC problems or resource quotas. I then verify node capacity and scheduling constraints rather than immediately changing application configuration.

Q: What does OOMKilled mean?

Answer: It means the container exceeded its available memory limit and was terminated. I check kubectl describe pod, kubectl top pod and kubectl top node, then compare usage with requests and limits and investigate application behavior, traffic changes and recent releases. I avoid simply increasing the limit without understanding the cause.

Q: How would you secure AKS?

Answer: I would use Azure/Kubernetes RBAC, managed or workload identity, private connectivity where required, network policies, Key Vault integration, image scanning and trusted registries, resource controls, pod security controls, monitoring/audit logs and a controlled upgrade strategy. Secrets should not be embedded in manifests or images.

Q: How would you troubleshoot AKS networking?

Answer: I work from Pod to Service to Endpoint to Ingress and then through DNS, NSG, routes, firewall and external dependencies. I use kubectl get/describe for Services and Ingress, inspect endpoints, and test DNS and HTTP connectivity from inside the Pod. I also compare the path with the intended VNet architecture.

Q: Production AKS application is down. What do you do?

Answer: First I establish impact—services, users and regions affected. I check ingress, Services, Pods, nodes and recent deployments. I review request rate, latency, errors, CPU/memory, Pod events and application logs. If a recent release aligns with the incident, I consider rollback. If capacity is the issue, I validate HPA, node capacity and autoscaling. After recovery I validate traffic and error rates, then complete RCA and preventive actions.

Q: How would you design AKS for production?

Answer: I would consider a private or appropriately secured cluster, dedicated subnets, multiple node pools, availability zones where supported, managed/workload identity, appropriate network design, ingress, private endpoints, monitoring, Prometheus/Grafana or Azure Monitor, HPA and Cluster Autoscaler, resource requests/limits, disruption budgets, upgrade strategy, security controls and Terraform-based provisioning with CI/CD.

5. Terraform / Infrastructure as Code

Q: Explain the Terraform workflow.

Answer: The normal workflow is terraform init, terraform validate, terraform plan, review/approval and terraform apply. For team environments I prefer remote state with locking, reusable modules, variables, controlled CI/CD execution and separation between environments.

Q: What is Terraform state?

Answer: Terraform state records the relationship between Terraform configuration and the actual infrastructure. It lets Terraform determine what exists and what changes are required. For teams, state should normally be stored in a secure remote backend with access control, locking and versioning.

Q: What is Terraform drift?

Answer: Drift occurs when infrastructure changes outside Terraform and no longer matches the Terraform configuration/state. terraform plan can identify differences, after which the team decides whether to import/reconcile the external change or restore the desired configuration.

Q: What is a Terraform module?

Answer: A module is reusable Terraform configuration. For example, an organization can create reusable modules for VNets, AKS, Key Vault and monitoring, then pass environment-specific variables from root modules.

Q: How do you manage production Terraform?

Answer: I keep production changes in source control, use remote state and locking, validate and plan through CI/CD, require review/approval for production applies, separate environments, protect secrets, use reusable modules and avoid manual changes that cause drift. I also maintain a rollback/recovery strategy appropriate to the resource type.

Q: What is a Terraform data source?

Answer: A data source reads information about existing infrastructure or external data without managing the lifecycle of that object. For example, Terraform can look up an existing resource group, subnet or subscription before creating dependent resources.

Q: What are variables and outputs?

Answer: Variables parameterize Terraform modules and configurations, while outputs expose useful values such as resource IDs, IP addresses or endpoints to other modules or automation.

6. Python and Automation

Q: How have you used Python for cloud automation?

Answer: I use Python primarily for automation and operational scripting. Typical use cases include API integration, configuration validation, operational data processing and repetitive cloud tasks. For Azure I can use the Azure SDK or REST APIs, authenticate with managed identity or service principals, implement logging and exception handling, and make the automation safe to rerun.

Q: How do you make Python automation production-ready?

Answer: I make it idempotent, validate inputs, use secure authentication, add structured logging and meaningful exit codes, implement timeouts and controlled retries, handle API errors, validate responses and test in non-production before controlled production use.

Q: List vs Tuple?

Answer: A list is mutable, so elements can be added, removed or changed. A tuple is immutable after creation. Lists are commonly used for collections that change; tuples are useful for fixed collections and can be hashable when their elements are hashable.

Q: List vs Set?

Answer: A list preserves order and can contain duplicates. A set stores unique elements and is optimized for membership testing. Sets are useful when uniqueness is the primary requirement.

Q: What is a Python dictionary?

Answer: A dictionary stores key-value pairs. It is useful for structured configuration and API data. Example: server = {'name': 'aks-prod', 'status': 'running'}.

Q: Explain exception handling.

Answer: Python uses try, except, else and finally. try contains code that may fail, except handles expected exceptions, else runs when no exception occurs, and finally executes regardless of success or failure. Production automation should catch specific exceptions and log enough context to diagnose failures.

Q: How do you call a REST API from Python?

Answer: I can use requests or an appropriate SDK. I set authentication securely, use a timeout, call raise_for_status or equivalent validation, parse JSON, handle expected HTTP errors, implement controlled retries where appropriate and log the request outcome without exposing secrets.

Q: What is idempotency?

Answer: An operation is idempotent when executing it multiple times produces the same desired end state as executing it once. For infrastructure automation this prevents duplicate resources or repeated changes from creating inconsistent state.

Q: How would you parse JSON in Python?

Answer: I can use the json module for strings/files or response.json() for HTTP responses. I validate that expected keys exist and handle malformed or unexpected data rather than assuming every response has the exact expected structure.

Q: How would you implement retries?

Answer: I retry only transient failures such as selected 429 or 5xx responses, use a bounded number of attempts, exponential backoff and jitter where appropriate, and avoid retrying permanent errors such as invalid authentication or malformed requests.

Q: What are *args and **kwargs?

Answer: *args captures additional positional arguments as a tuple, while **kwargs captures additional keyword arguments as a dictionary. They are useful when designing flexible functions, but I use explicit parameters when a stable interface is preferable.

Q: What is a generator?

Answer: A generator yields values lazily instead of creating the complete collection in memory. It is useful for processing large datasets or streams efficiently.

Q: What is a decorator?

Answer: A decorator wraps a function or class to add behavior without modifying its core implementation. Common uses include logging, timing, authentication and retries.

7. Azure DevOps / CI-CD

Q: Explain your CI/CD pipeline.

Answer: CI starts with source control, followed by build, unit tests, static analysis and security checks, and then publishes an immutable artifact. CD deploys that artifact to the target environment, with Terraform managing infrastructure where applicable. Production should use approvals, controlled rollout, health checks and rollback procedures, followed by monitoring and validation.

Q: How are tools integrated in a CI/CD pipeline?

Answer: Git triggers the pipeline. Build tools compile or package code. Testing and static/security scanning run as quality gates. Terraform can provision infrastructure. Docker builds and pushes images to a registry such as ACR. Helm or deployment tooling deploys to AKS. Azure Monitor/Prometheus/Grafana provide post-deployment validation and alerts.

Q: Blue-Green vs Canary deployment?

Answer: Blue-Green maintains two environments: current and new. After validating the new environment, traffic is switched. Canary gradually sends a small percentage of traffic to the new version and increases it based on health signals. Canary reduces blast radius; Blue-Green provides a simpler full cutover and rollback model.

Q: What is rollback?

Answer: Rollback restores the previous known-good application or infrastructure state after a failed change. For applications, this can mean redeploying a previous immutable artifact or Helm release. For infrastructure, rollback must be handled carefully because not every resource change is safely reversible.

8. SRE / Reliability

Q: What is an SLI?

Answer: An SLI is a quantitative indicator of service behavior, such as successful request percentage, latency or availability.

Q: What is an SLO?

Answer: An SLO is the target level for an SLI, such as 99.9% successful requests over a defined measurement window.

Q: What is an SLA?

Answer: An SLA is a contractual or customer-facing commitment that can include availability and service obligations. It is distinct from an internal SLO used for engineering reliability management.

Q: What is an error budget?

Answer: If an SLO is 99.9%, the allowed unreliability is 0.1% over the measurement period. That allowance is the error budget. Teams can use it to balance reliability work and release velocity.

Q: What are the Four Golden Signals?

Answer: Latency, Traffic, Errors and Saturation. Together they provide a practical starting point for monitoring service health.

Q: What is alert fatigue?

Answer: Alert fatigue occurs when engineers receive too many noisy or low-value alerts, making important alerts harder to notice. I reduce it by alerting on symptoms and actionable thresholds, tuning sensitivity, grouping related alerts and using SLO-based or impact-based alerting where appropriate.

Q: How do you handle a production incident?

Answer: I establish impact and severity, assign or clarify ownership, collect metrics/logs/traces, isolate the fault domain, mitigate quickly, communicate status, validate recovery, then conduct RCA and implement preventive actions. The goal during an incident is first to restore service safely, then understand and prevent recurrence.

Q: What is MTTR?

Answer: Mean Time to Recovery or Repair measures the average time taken to restore service after an incident, depending on the organization's exact definition. Reducing MTTR typically involves better detection, runbooks, automation, rollback mechanisms and incident processes.

Q: What is capacity planning?

Answer: Capacity planning estimates future resource requirements based on traffic, workload growth, performance, resource utilization, scaling limits and business forecasts. In AKS it includes node capacity, Pod requests, autoscaler behavior, IP address space and dependency capacity.

9. Production Scenarios

Q: CPU is 95% in production. What do you do?

Answer: I determine whether the high CPU is isolated to a Pod, node or cluster. I compare usage against requests and limits, check traffic and recent deployments, identify the top consumers and determine whether the increase is expected or abnormal. If legitimate, I validate HPA and node capacity; if abnormal, I investigate application behavior and recent changes. I mitigate through appropriate scaling or rollback and then tune resources or autoscaling based on evidence.

Q: AKS cannot access Azure Key Vault. What do you check?

Answer: I check which identity the workload is using, whether the identity is correctly configured, whether it has the required RBAC/data-plane permissions, and whether Key Vault firewall/private endpoint/private DNS settings allow connectivity. I test DNS and network connectivity from the Pod and inspect application and Azure audit logs.

Q: A new application must be onboarded into Azure. What do you consider?

Answer: I consider subscription/resource boundaries, management groups, RBAC, policy, tags, VNet/subnet design, NSGs, routes, DNS, private endpoints, firewall controls, identity, Key Vault, monitoring, backup/DR and Terraform. For AKS I also define node pools, ingress, scaling, observability and upgrade requirements. I automate the foundation and connect it to CI/CD before production onboarding.

Q: What happens when a user types a URL for an application?

Answer: At a high level the client resolves DNS, connects to the public or private endpoint, and reaches a load balancer, Application Gateway or ingress layer. The request is routed to a Kubernetes Service and then to a Pod. In a private architecture, private DNS can resolve the hostname to a private

endpoint or internal load-balancing address.

Q: How would you troubleshoot a failed deployment?

Answer: I check pipeline logs first to identify whether the failure is build, test, artifact, infrastructure or deployment related. Then I validate the exact version, credentials/identity, target environment, Terraform plan/apply output, Helm or kubectl events, Pod status and application health. I compare with recent successful deployments and rollback when a bad release is the confirmed cause.

Q: How would you approach a P1 incident?

Answer: I first protect customer impact and establish a clear incident owner. I quickly determine blast radius, review the most relevant dashboards, logs and recent changes, and split investigation into likely domains such as application, infrastructure, network or dependency. I communicate concise updates, mitigate using the safest available action, validate recovery and then complete a blameless RCA with concrete prevention items.

10. Complete Rapid-Fire Question Bank — All Questions Answered

Azure

1. What is a VNet?

Answer: A logically isolated network in Azure that provides IP addressing, subnets, routing and connectivity for resources.

2. What is a subnet?

Answer: A subdivision of a VNet address space used to segment workloads and apply network controls.

3. What is an NSG?

Answer: A set of inbound/outbound network security rules applied at subnet or NIC level.

4. What is a route table?

Answer: A collection of routes that determines how traffic is forwarded from a subnet.

5. What is VNet peering?

Answer: Private connectivity between VNets over the Azure backbone; it is not automatically transitive.

6. What is a Private Endpoint?

Answer: A private IP-based connection from a VNet to an Azure PaaS service using Private Link.

7. Private Endpoint vs Service Endpoint?

Answer: Private Endpoint gives the service a private IP in the VNet; Service Endpoint extends VNet identity to a public service endpoint.

8. What is Azure Firewall?

Answer: A managed centralized network security service for controlling and inspecting traffic.

9. What is Application Gateway?

Answer: A Layer-7 HTTP/HTTPS load balancer with features such as URL/host routing and WAF integration.

10. What is WAF?

Answer: Web Application Firewall protection for HTTP/HTTPS applications against common web attacks.

11. What is ExpressRoute?

Answer: Private connectivity between on-premises and Azure through an ExpressRoute provider.

12. What is VPN?

Answer: Encrypted network connectivity, commonly site-to-site, between on-premises and Azure over the internet.

13. What is Azure Policy?

Answer: Governance service used to audit or enforce resource configuration standards.

14. What is Azure RBAC?

Answer: Role-based authorization that controls who can perform actions on Azure resources at a defined scope.

15. What is Managed Identity?

Answer: Azure-managed identity used by resources/applications to authenticate without storing credentials.

16. What is Key Vault?

Answer: Managed service for securely storing secrets, keys and certificates.

17. What is a Management Group?

Answer: A governance hierarchy above subscriptions used to apply policy and RBAC at scale.

18. What is a Subscription?

Answer: An Azure resource and billing boundary that can contain resource groups and resources.

19. What is a Resource Group?

Answer: A logical container for Azure resources that commonly share a lifecycle.

20. What is an Availability Zone?

Answer: A physically separate datacenter location within an Azure region used for resilience.

AKS / Kubernetes

21. What is AKS architecture?

Answer: Managed Kubernetes control plane plus customer-managed worker nodes/workloads, with Azure integrations for networking, identity, storage and monitoring.

22. What is a Pod?

Answer: The smallest deployable Kubernetes unit, containing one or more containers sharing network/storage context.

23. What is a Deployment?

Answer: A controller for managing stateless replicated Pods and rolling updates.

24. What is a Service?

Answer: A stable network abstraction that routes traffic to selected Pods.

25. What is Ingress?

Answer: HTTP/HTTPS routing into Kubernetes Services through an ingress controller.

26. What is a ConfigMap?

Answer: A Kubernetes object for non-sensitive configuration data.

27. What is a Secret?

Answer: A Kubernetes object intended for sensitive configuration data; secure external secret management is often preferable for production.

28. What is a DaemonSet?

Answer: A controller that runs a Pod on each eligible node, commonly for agents such as logging or monitoring.

29. What is a StatefulSet?

Answer: A controller for stateful workloads needing stable identity and persistent storage semantics.

30. Deployment vs StatefulSet?

Answer: Deployment is primarily stateless; StatefulSet provides stable identity, storage and ordering for stateful workloads.

31. ClusterIP vs LoadBalancer?

Answer: ClusterIP exposes a Service internally; LoadBalancer integrates with the cloud provider to expose it externally.

32. What is HPA?

Answer: Horizontal Pod Autoscaler changes the number of Pods based on metrics.

33. What is Cluster Autoscaler?

Answer: Adds or removes worker nodes based on scheduling demand and capacity.

34. What are taints/tolerations?

Answer: Taints repel workloads from nodes; tolerations allow matching workloads to schedule there.

35. What is node affinity?

Answer: Scheduling rules that prefer or require Pods to run on nodes with specific labels.

36. What is CrashLoopBackOff?

Answer: A Pod/container repeatedly fails and Kubernetes backs off between restart attempts.

37. What is ImagePullBackOff?

Answer: Kubernetes cannot pull the container image and is backing off before retrying.

38. Why is a Pod Pending?

Answer: Common causes include insufficient resources, scheduling constraints, taints, affinity, PVC issues or quotas.

39. What is OOMKilled?

Answer: A container exceeded its memory limit and was terminated.

40. Readiness vs Liveness?

Answer: Readiness controls traffic eligibility; liveness determines whether the container should be restarted when unhealthy.

Terraform

41. What is Terraform state?

Answer: The record Terraform uses to map configuration to managed infrastructure.

42. What is a remote backend?

Answer: A remote location for storing Terraform state, enabling team collaboration and controlled access.

43. What is state locking?

Answer: A mechanism preventing concurrent state modifications that could corrupt or conflict with Terraform operations.

44. What is a module?

Answer: Reusable Terraform configuration packaged for consistent infrastructure patterns.

45. What is a variable?

Answer: An input parameter used to make Terraform configuration reusable.

46. What is an output?

Answer: A value exposed by a Terraform module or configuration for use by other automation/configurations.

47. What is a data source?

Answer: A read-only lookup of existing resources or external information.

48. What is Terraform drift?

Answer: A mismatch between the declared/managed Terraform configuration and actual infrastructure caused by external changes.

49. What does terraform plan do?

Answer: Calculates and displays proposed infrastructure changes without applying them.

50. What does terraform apply do?

Answer: Applies the approved Terraform changes to infrastructure.

51. What does terraform destroy do?

Answer: Attempts to remove resources managed by the selected Terraform configuration; it requires careful production controls.

52. How should production Terraform be managed?

Answer: Through source control, remote state, locking, CI/CD plans, approvals, least privilege and controlled applies.

Python

53. List vs tuple?

Answer: List is mutable; tuple is immutable.

54. List vs set?

Answer: List preserves order and allows duplicates; set stores unique elements and is efficient for membership tests.

55. Dictionary?

Answer: A key-value data structure used for structured data and configuration.

56. Exception handling?

Answer: Use try/except/else/finally and catch specific expected exceptions with useful logging.

57. Function?

Answer: A reusable block of code that can accept parameters and return values.

58. Lambda?

Answer: A small anonymous function expression, useful for concise transformations or callbacks.

59. Generator?

Answer: A lazy iterator that yields values one at a time, reducing memory use for large sequences.

60. Decorator?

Answer: A wrapper that adds behavior to a function or class without changing its core implementation.

61. *args / **kwargs?

Answer: *args captures extra positional arguments; **kwargs captures extra keyword arguments.

62. REST API call?

Answer: Use an HTTP client/SDK with secure authentication, timeout, response validation, exception handling and appropriate retries.

63. JSON parsing?

Answer: Use json.loads/json.load or response.json(), then validate expected fields and handle malformed data.

64. File handling?

Answer: Use context managers such as with open(...) to ensure files are closed and handle encoding/errors appropriately.

65. Logging?

Answer: Use Python's logging framework with levels, structured context where possible, and never log secrets.

66. Retry?

Answer: Retry only transient failures with bounded attempts, exponential backoff/jitter and no retries for permanent errors.

67. How do you make automation idempotent?

Answer: Design operations around desired state and check whether the desired state already exists before making changes.

SRE

68. SLI?

Answer: A measured indicator of service behavior such as availability, latency or successful request rate.

69. SLO?

Answer: A target value for an SLI over a defined period.

70. SLA?

Answer: A contractual/customer-facing service commitment.

71. Error budget?

Answer: The amount of unreliability permitted by an SLO during its measurement window.

72. Four Golden Signals?

Answer: Latency, Traffic, Errors and Saturation.

73. Incident management?

Answer: A structured process to detect, triage, mitigate, communicate, recover and learn from production incidents.

74. RCA?

Answer: Root Cause Analysis identifies contributing technical/process causes and preventive actions after an incident.

75. Postmortem?

Answer: A documented review of an incident covering impact, timeline, causes, response and prevention actions.

76. Alert fatigue?

Answer: Excessive noisy alerts that reduce attention to actionable incidents.

77. Capacity planning?

Answer: Forecasting and provisioning resources to meet expected workload, performance and availability requirements.

78. Availability?

Answer: The proportion of time or requests for which a service is usable according to a defined measurement.

79. MTTR?

Answer: Mean Time to Recovery/Repair, measuring average time to restore service after incidents depending on the organization's definition.

11. Architecture and Troubleshooting Cheat Sheets

Enterprise Azure Landing Zone

```
Azure Tenant
→ Management Groups
→ Platform / Workload Subscriptions
→ Hub VNet + Shared Connectivity
→ Spoke VNets + Application Workloads
→ Azure Firewall / VPN / ExpressRoute / DNS
→ RBAC + Azure Policy + Tags + Diagnostics
```

Production AKS

```
VNet/Subnet
→ AKS
→ System + Application Node Pools
→ Pods / Deployments
→ Services
→ Ingress / Application Gateway
→ Private Endpoints / PaaS Dependencies
→ Azure Monitor / Prometheus / Grafana
→ HPA + Cluster Autoscaler
→ Terraform + CI/CD
```

AKS troubleshooting path

```
Pod
→ Container logs/events
→ Service
→ Endpoints
→ Ingress
→ DNS
→ NSG
→ Route Table
→ Firewall
→ VNet
→ External Dependency
```

CI/CD flow

```
Git
→ Build
→ Unit Tests
→ Security / Quality Gates
→ Artifact
→ Terraform Plan
→ Approval
→ Terraform Apply
→ Deployment
→ Smoke Test
→ Monitoring
→ Rollback if required
```

SRE flow

```
SLI
→ SLO
→ Error Budget
→ Monitoring
→ Alerting
→ Incident
→ Mitigation
→ Recovery
→ RCA
→ Preventive Actions
```

12. Final Night-Before Checklist

☐ Explain Landing Zone without memorized buzzwords.

☐ Draw hub-spoke architecture from memory.

☐ Explain Management Group → Subscription → Resource Group hierarchy.

☐ Know Policy vs RBAC.

☐ Know NSG vs Firewall vs WAF.

☐ Know Private Endpoint vs Service Endpoint.

☐ Explain AKS control plane, node pools, Pods, Services and Ingress.

☐ Troubleshoot CrashLoopBackOff, ImagePullBackOff, Pending and OOMKilled.

☐ Explain HPA and Cluster Autoscaler.

☐ Explain Terraform state, remote backend, modules and drift.

☐ Write basic Python without looking up syntax.

☐ Explain Python API automation and idempotency.

☐ Explain CI/CD stages and production approvals.

☐ Know SLI/SLO/SLA/Error Budget and Four Golden Signals.

☐ Prepare two real production incidents from your own experience.

☐ Be precise about what you personally implemented versus what you supported or understand conceptually.

Interview answer formula: Direct answer → Why it matters → Practical example → Production consideration.

Important: Use your actual project experience. If the interviewer asks for a specific implementation you have not personally performed, say so and explain how you would implement it rather than inventing experience.